

Projeto UpCar

Oct 8, 2026 · Trabalho realizado por Luan Henrique, Lorenzo Vanelli, Murilo Opis

1. Visão geral

O UpCar2 é um aplicativo Android de caronas universitárias: o passageiro busca motoristas com destino ao campus, negocia o valor por chat e confirma a carona. O repositório é [Lorenzovanelli/UpCar2](#) e possui um único commit ("projeto base com as telas e navegacao").

O app funciona como protótipo: todos os dados ficam em memória e a resposta do motorista no chat é simulada. Não há backend, banco de dados nem login.

Funcionalidades implementadas:

- Mapa (OpenStreetMap) com pinos dos motoristas e do campus.
- Busca por destino, que filtra a lista de motoristas.
- Cadastro de nova oferta de carona, com validação de placa, horário, preço e coordenadas.
- Remoção de motorista (segurar o card) e de mensagem própria (segurar o balão).
- Chat por carona, com contraproposta rápida de valor.
- Tela de carona confirmada, com resumo do motorista, trajeto e valor combinado.

2. Tecnologias e dependências

O projeto é escrito em Kotlin, com interface em Jetpack Compose (Material 3). As versões abaixo foram lidas de `app/build.gradle.kts` e `gradle/libs.versions.toml`.

Item	Valor / versão	Uso
Android Gradle Plugin	9.3.3	Build do app
Kotlin	2.2.10	Linguagem e plugin do Compose
Compose BOM	2026.02.01	Versões do Compose e Material 3
compileSdk / targetSdk	37	Nível de API
minSdk	24	Android 7.0 ou superior
Java	11	Compatibilidade de código
navigation-compose	2.9.8	Navegação entre telas

Item	Valor / versão	Uso
osmdroid-android	6.1.20	Mapa OpenStreetMap
material-icons-core	via BOM	Ícones (Search, Check, ArrowBack)
lifecycle-viewmodel-compose	X.Y.Z (placeholder)	<code>viewModel()</code> nas telas, veja a seção 9
activity-compose	1.8.0	<code>setContent</code> e <code>edge-to-edge</code>
JUnit / Espresso	4.13.2 / 3.5.1	Testes (apenas os exemplos gerados)

Identificadores: `applicationId` e `namespace` são `com.example.upcar`, versão 1.0 (`versionCode` 1).

3. Arquitetura e estrutura de arquivos

O app tem uma única `Activity` e um único `CaronaViewModel`, compartilhado por todas as telas. O `ViewModel` guarda três listas observáveis (`mutableStateListOf`): motoristas, caronas e mensagens. As telas leem essas listas e chamam os métodos do `ViewModel` para alterá-las. Todo o código está em `app/src/main/java/com/example/upcar/`.

Arquivo	Responsabilidade
<code>MainActivity.kt</code>	Configura o osmdroid, ativa <code>edge-to-edge</code> e exibe <code>AppNavigation</code> dentro do tema
<code>AppNavigation.kt</code>	Define as rotas e o <code>NavHost</code> ; cria o <code>ViewModel</code> compartilhado
<code>CaronaViewModel.kt</code>	Estado e regras de negócio (solicitar, contraproposta, confirmar, remover)
<code>Modelos.kt</code>	Classes <code>Motorista</code> , <code>Carona</code> , <code>Mensagem</code> e constantes do campus
<code>Dados.kt</code>	Três motoristas iniciais de exemplo
<code>ProcuraMotorista.kt</code>	Tela principal: busca, mapa, lista de cards e formulário de oferta
<code>MapaMotoristas.kt</code>	Mapa osmdroid integrado ao Compose, com pinos em forma de rótulo

Arquivo	Responsabilidade
Chat.kt	Tela de conversa, contraproposta e botão de confirmação
CaronaConfirmada.kt	Tela de resumo da carona confirmada
ComponentesComuns.kt	BarraTopo , Avatar e TelaNaoEncontrada
Validacoes.kt	Validação e normalização de horário e placa
Formatadores.kt	Formatação de valores em reais e de notas
ui/theme/	Tema Material 3 e objeto Cores com a paleta do app

O destino padrão é "Campus Centro • Bloco B" (DESTINO_PADRAO), e o campus fica em latitude -25.4284 e longitude -49.2733 (região de Curitiba).

4. Modelos de dados

Três `data class` em `Modelos.kt` representam todo o estado do app. Uma `Carona` aponta para um `Motorista` por `motoristaId`, e uma `Mensagem` aponta para uma `Carona` por `caronaId`.

Motorista

Campo	Tipo	Descrição
id	Int	Identificador
nome	String	Nome do motorista
nota	Double	Avaliação (novos motoristas começam com 5.0)
horario	String	Horário de saída, formato HH:mm
preco	Double	Preço inicial da carona
corridas	Int	Quantidade de corridas (0 nos cadastrados pelo formulário)
carro, placa	String	Modelo e cor, e placa do veículo
origem, destino	String	Ponto de embarque e destino
lat, lng	Double	Posição do pino no mapa

Carona

Campo	Tipo	Descrição
id, motoristald	Int	Identificador e motorista associado
origem, destino, horario	String	Cópia dos dados do motorista no momento da solicitação
valor	Double	Valor atual, alterado pela contraproposta
confirmada	Boolean	<code>true</code> após a confirmação (padrão <code>false</code>)
aguardandoResposta	Boolean	<code>true</code> enquanto a resposta simulada não chega (padrão <code>false</code>)

Mensagem

Campo	Tipo	Descrição
id, caronald	Int	Identificador e carona associada
texto, horario	String	Conteúdo e hora de envio (HH:mm)
minha	Boolean	<code>true</code> se foi enviada pelo passageiro

5. Navegação e telas

O fluxo tem três telas em sequência, definidas em `AppNavigation.kt`. As rotas de chat e de confirmação recebem o argumento inteiro `caronaId`.

1. **Motoristas** (`motoristas` , tela inicial): o passageiro busca e escolhe um motorista.
2. **Chat** (`chat/{caronaId}`): negociação do valor com o motorista.
3. **Carona confirmada** (`confirmada/{caronaId}`): resumo final. O botão "Voltar ao início" retorna à tela de motoristas.

Tela de motoristas



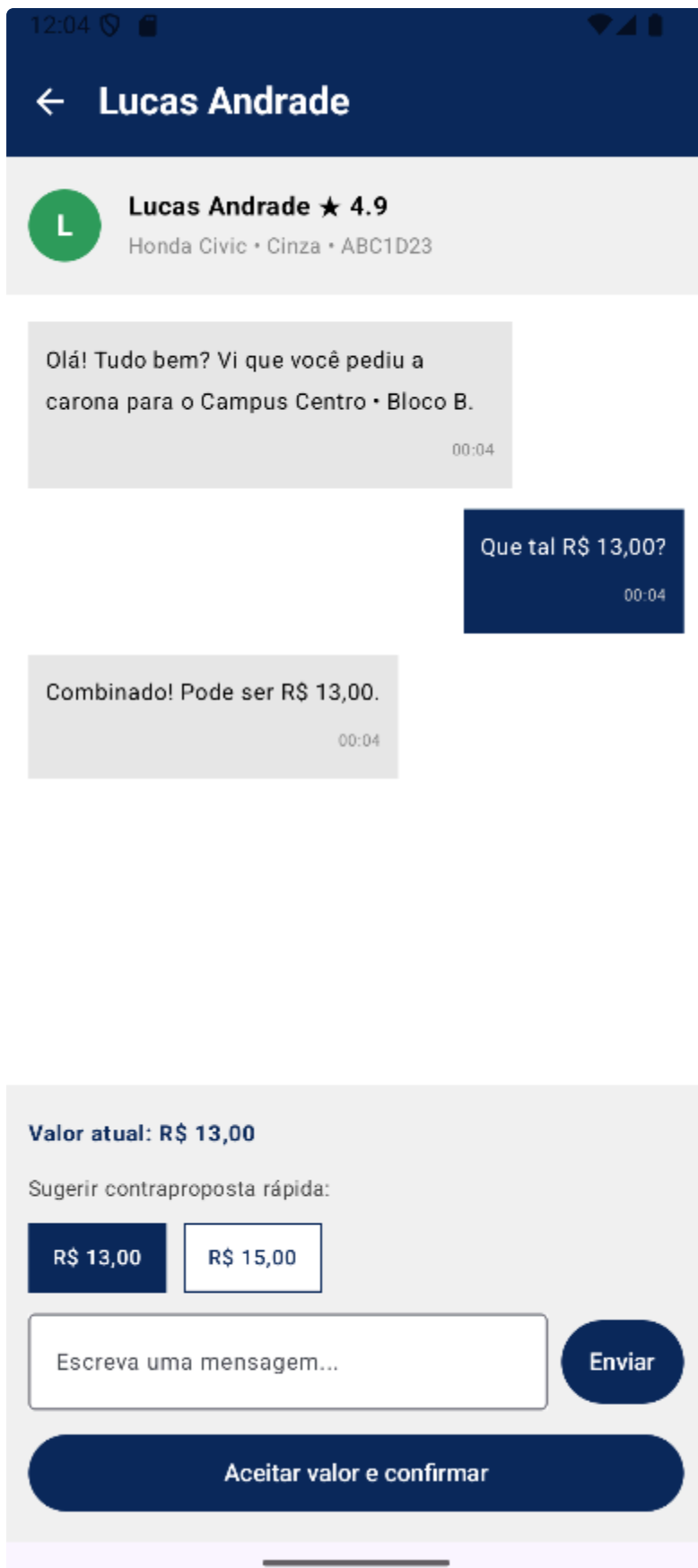
O topo mostra o mapa com um campo "Para:" sobreposto, preenchido com "Campus Centro". A parte de baixo (55%) lista os motoristas cujo destino contém o texto buscado, sem diferenciar maiúsculas. Tocar em um card ou em um pino seleciona o motorista: o pino

fica vermelho, o mapa centraliza nele e a lista rola até o card. O botão "Pedir" mostra um aviso e abre o chat. "+ Oferecer carona" abre o formulário, e segurar um card pede confirmação para remover o motorista.

Mapa

`MapaMotoristas` envolve um `MapView` do osmdroid em `AndroidView`, com tiles Mapnik, zoom inicial 13 e gestos multitoque. O campus tem pino verde, os motoristas pino azul e o selecionado pino vermelho. Os rótulos são bitmaps desenhados em código, com cache, e mostram o primeiro nome e a hora (ex.: "Lucas (18h)"). O mapa acompanha o ciclo de vida da tela (`onResume`, `onPause`, `onDetach`).

Chat



Mostra o cabeçalho do motorista, as mensagens, o valor atual e as opções de contraproposta: o preço do motorista menos R\$ 2,00 (mínimo de R\$ 1,00) e o preço original. Há também campo de texto livre. Segurar uma mensagem própria a remove. O

botão principal diz "Aceitar valor e confirmar" e passa a "Ver carona confirmada" depois da confirmação.

Carona confirmada



Exibe um ícone de sucesso, os dados do motorista (nome, nota, carro e placa) e o resumo com embarque, destino, saída e valor combinado. Se a carona ou o motorista não existirem, `TelaNaoEncontrada` é exibida.

6. Regras de negócio

Todas as regras estão em `CaronaViewModel.kt`.

Operação	Comportamento
<code>solicitarCarona</code>	Se já existe carona não confirmada com o mesmo motorista, reaproveita essa. Senão, cria a carona com origem, destino, horário e preço do motorista e adiciona uma mensagem de saudação do motorista.
<code>enviarContraproposta</code>	Ignora se a carona já está confirmada ou aguardando resposta. Atualiza o valor, marca <code>aguardandoResposta</code> , envia "Que tal R\$ X?" e, após 1,5 s, libera a carona e simula a resposta "Combinado!" do motorista.
<code>confirmarCarona</code>	Ignora se já confirmada ou aguardando resposta. Marca <code>confirmada</code> e envia a mensagem de fechamento com o valor.
<code>enviarMensagem</code>	Cria a mensagem com a hora local (HH:mm). O padrão é mensagem do passageiro.
<code>adicionarMotorista</code>	Cria motorista com nota 5.0 e 0 corridas. Sem coordenadas, o pino é posicionado de forma determinística perto do campus, a partir do id.
<code>removerMotorista</code>	Remove também as caronas e mensagens desse motorista.

O motorista nunca recusa: a contraproposta é sempre aceita. Enquanto aguarda a resposta, os botões de valor e o botão de confirmação ficam desabilitados.

7. Validações do formulário

O formulário "Oferecer carona" verifica os campos em sequência (nome, carro, placa, saída, preço, embarque, coordenadas) e mostra um aviso (`Toast`) no primeiro erro encontrado. As regras de placa e horário ficam em `Validacoes.kt`.

Campo	Regra	Mensagem de erro
Nome, carro, ponto de embarque	Não pode ficar em branco	"Informe o nome do motorista." e equivalentes
Placa	Formato Mercosul AAA9A99 (<code>^[A-Z]{3}[0-9][A-Z0-9][0-9]{2}\$</code>); aceita hífen e minúsculas, que são normalizados	"Placa inválida. Exemplo: ABC1D23."
Saída	<code>H:mm</code> ou <code>HH:mm</code> , de 00:00 a 23:59; normalizado para <code>HH:mm</code>	"Saída inválida. Use o formato 18:30."
Preço	Número maior que zero; aceita vírgula decimal	"Informe um preço válido."
Latitude e longitude	Opcionais, mas juntas: latitude de -90 a 90 e longitude de -180 a 180	"Coordenadas inválidas. Informe latitude e longitude, ou deixe ambas em branco."

O campo de placa converte o texto para maiúsculas e limita a 8 caracteres. A placa no formato antigo (`AAA-9999`) não é aceita pela expressão regular.

8. Como executar

O projeto usa o Gradle Wrapper e não precisa de chaves de API. O mapa usa os tiles públicos do OpenStreetMap, então o aparelho precisa de internet.

1. Clone o repositório: `git clone https://github.com/Lorenzovaneli/UpCar2.git` .
2. Abra a pasta no Android Studio, com uma versão compatível com o Android Gradle Plugin 9.3.3, e aguarde a sincronização do Gradle.
3. Escolha um emulador ou celular com Android 7.0 (API 24) ou superior e execute o módulo `app` .

Na linha de comando, o equivalente é `./gradlew assembleDebug` , que gera o APK de depuração.

9. Pontos de atenção encontrados no código

Três itens impedem o app de compilar ou abrir como está no repositório. Foram identificados pela leitura do código; o app não foi compilado nem executado para esta documentação.

Item	Onde	Efeito	Correção sugerida
Versão placeholder X.Y.Z	app/build.gradle.kts, dependência lifecycle-viewmodel-compose	O Gradle não resolve a dependência e o build falha	Informar uma versão real ou mover a dependência para libs.versions.toml
Nome da Activity diferente	AndroidManifest.xml usa com.example.testeupcar.MainActivity, mas a classe é com.example.upcar.MainActivity	O Android não encontra a classe ao abrir o app	Trocar por .MainActivity ou pelo nome completo correto
Sem permissão de internet	AndroidManifest.xml	O osmdroid não baixa os tiles e o mapa fica sem imagem	Adicionar android.permission.INTERNET
README vazio	README.md contém só o título "UpCar2"	Quem abre o repositório não sabe o que o app faz	Usar o resumo e a seção 8 deste documento
Dados voláteis	CaronaViewModel	Motoristas cadastrados, caronas e mensagens somem quando o processo é encerrado	Persistir com Room ou um backend
Configuração do osmdroid duplicada	MainActivity.kt e MapaMotoristas.kt	Sem efeito visível; o carregamento ocorre duas vezes, com arquivos de preferências diferentes	Manter em um único lugar
Otimização desligada no release	optimization { enable = false }	O APK de release não é reduzido nem ofuscado	Ativar antes de publicar
Testes apenas de exemplo	ExampleUnitTest e ExampleInstrumentedTest	As regras do ViewModel e as validações não têm cobertura	Testar Validacoes.kt e CaronaViewModel